TATA TECHNOLOGIES | INNOVENT-27 | AI AT THE EDGE | AEROSPACE TRACK

AeroTwin — Use Case & User Story Document

Real-World Scenarios, Actor Profiles & Acceptance Criteria | Team FELONS | InnoVent-27

PURPOSE OF THIS DOCUMENT

This document defines the real-world use cases and user stories that AeroTwin is designed to address. Each use case describes a concrete scenario from the perspective of an actual aviation MRO stakeholder — a maintenance engineer, an airline operations manager, or a safety officer — and maps it to the specific AeroTwin feature that solves their problem. This document bridges the gap between AeroTwin's technical architecture and its real-world human value.

SYSTEM ACTORS — WHO USES AEROTWIN

AeroTwin serves three primary user personas, each with distinct goals and pain-points:

▸ Actor 1: Line Maintenance Engineer

Profile: Rajesh Sharma, 34. Works in the MRO hangar at Delhi Indira Gandhi International Airport. Responsible for the day-to-day inspection and sign-off of engine components between flights. Currently relies on paper-based inspection logs and a scheduled check list. Has no real-time visibility into how a component is degrading between his inspection cycles.

AeroTwin features used: Real-time health dashboard, anomaly alerts, historical sensor log, component-level drill-down.

▸ Actor 2: Airline Fleet Operations Manager

Profile: Priya Menon, 42. Head of Technical Operations at a regional Indian carrier. Responsible for scheduling maintenance windows across a 40-aircraft fleet without grounding too many planes at once. Currently works from fixed maintenance schedules that frequently result in either over-maintenance (healthy planes grounded) or missed faults.

AeroTwin features used: Fleet-level health overview, RUL-based maintenance scheduling, cost-impact projections.

▸ Actor 3: Aviation Safety Officer

Profile: Wing Commander (Retd.) Arun Verma, 51. Responsible for audit and compliance at a regional carrier. Needs a verifiable, tamper-evident record of all engine health events and maintenance actions taken. Currently reviews printed log books after incidents rather than having proactive visibility into developing safety risks.

AeroTwin features used: Maintenance log audit trail, anomaly event history, severity escalation records.

DETAILED USE CASES

▶ UC-01: Real-Time Engine Health Monitoring During Simulated Flight

Actor: Line Maintenance Engineer (Rajesh Sharma)	**Trigger:** Rajesh opens the AeroTwin dashboard at the start of his shift to check the health of the engines currently in flight.
Precondition: AeroTwin dashboard is open. The simulation engine is running and streaming sensor data. Flight AI-302 (Delhi to London) is in progress.	**AeroTwin Features:** Live sensor dashboard, Three.js component visualization, real-time Recharts trend graphs, RUL panel.

Main Flow:

1. Rajesh opens the AeroTwin dashboard in his browser.
2. The dashboard displays three engine components (turbine blade, bearing, compressor) in real-time, all currently GREEN (healthy).
3. Live sensor readings stream in — temperature: 542°C, vibration: 1.8mm/s, RPM: 9,450.
4. The fatigue trend graphs show a gradual, expected upward trend across all three components.
5. The RUL panel shows: Turbine Blade — 720 hrs remaining, Bearing — 680 hrs remaining, Compressor — 890 hrs remaining.
6. The system shows 'No active alerts. All components operating within normal parameters.'
7. Rajesh is confident the aircraft can complete its next three scheduled routes without requiring an unplanned inspection.

Alternate Flow: If the bearing's vibration reading spikes unexpectedly (see UC-03), the dashboard immediately flags an anomaly and Rajesh receives an alert.

Postcondition: Rajesh has a clear, real-time picture of engine health without needing to wait for a physical inspection or a post-flight data download.

VALUE DELIVERED

Replaces a 'no news is good news' monitoring model with continuous, proactive visibility. Rajesh can spot a developing issue during the flight, not after landing.

▶ UC-02: Predictive Maintenance Scheduling Based on RUL

Actor: Fleet Operations Manager (Priya Menon)	**Trigger:** The AeroTwin alert engine generates an AMBER alert: 'Bearing on Engine #4 — Schedule inspection within 340 flight hours.'
Precondition: AeroTwin has been running for the equivalent of 200 simulated flight hours. The bearing's health has declined to 71% and its predicted RUL is 340 hours.	**AeroTwin Features:** RUL prediction panel, AMBER alert, maintenance recommendation engine, maintenance log.

Main Flow:

1. Priya receives the AMBER alert notification on her dashboard.
2. She clicks the bearing component on the Three.js visualisation to drill into its detail view.
3. The detail view shows: current health 71%, fatigue trend accelerating slightly, RUL prediction: 340 hrs (confidence: 89%).
4. The system recommends: 'Schedule bearing inspection and potential replacement within the next maintenance window — approximately 2 weeks at current flight cycle.'
5. Priya checks the fleet schedule and identifies a 6-hour ground window for Engine #4's aircraft in 10 days.
6. She schedules the bearing inspection for that window — before the RUL threshold is breached.
7. AeroTwin logs the scheduling decision in the maintenance_log table.

Alternate Flow: If no suitable ground window is available within the RUL window, the severity automatically escalates to RED and Priya is prompted to consider fleet substitution.

Postcondition: A maintenance action is scheduled proactively, weeks before a failure could occur, with zero unplanned downtime.

VALUE DELIVERED

Transforms maintenance scheduling from a fixed-calendar process to a data-driven, RUL-based model — eliminating both over-maintenance and under-maintenance scenarios.

UC-03: Real-Time Anomaly Detection and Emergency Alert

Actor: Line Maintenance Engineer (Rajesh Sharma)	**Trigger:** The injected anomaly causes turbine blade vibration to jump from 3.2mm/s to 11.7mm/s in a single simulation tick — a z-score of 4.1, well above the 3.5 detection threshold.
Precondition: AeroTwin is monitoring a flight at 280 simulated flight hours. All components are AMBER (degrading normally). A sudden bird-strike event causes a step-change in turbine blade vibration.	**AeroTwin Features:** Z-score anomaly detector, CRITICAL alert, Three.js visual flash, RUL re-inference on anomaly, maintenance log write.

Main Flow:

1. The anomaly z-score detector flags the vibration spike in the same simulation tick it occurs.
2. Within 2 seconds, a CRITICAL alert fires: ' CRITICAL — Turbine Blade: Anomalous vibration spike detected. Possible foreign object damage. GROUND AIRCRAFT. Immediate inspection required.'
3. The turbine blade mesh on the Three.js visualisation flashes RED and pulses.
4. The alert panel at the top of the dashboard shows the critical alert with a timestamp, component ID, and recommended action.
5. The RUL model is triggered to re-run immediately on the post-spike sensor state and returns a new estimate: 'RUL: < 20 hours — CRITICAL.'
6. Rajesh immediately contacts the airline operations team to divert or ground the aircraft.

7. The full event — pre-anomaly sensor history, spike moment, alert generation, and recommended action — is written to the maintenance_log table for the safety officer's audit.

Alternate Flow: If the spike is transient (returns to normal in the next 3 ticks), the system downgrades the alert to AMBER with a note: 'Transient anomaly — monitor closely.'

Postcondition: A sudden, dangerous fault is detected within 2 seconds of occurrence. A maintenance action recommendation is generated automatically — compared to hours or days of delay in a traditional periodic log review model.

VALUE DELIVERED

Demonstrates the core safety value of continuous real-time monitoring: catching a sudden, life-threatening fault instantly rather than discovering it post-flight or post-incident.

▶ UC-04: Historical Trend Review for Root Cause Analysis

Actor: Aviation Safety Officer (Wing Cdr. (Retd.) Arun Verma)	**Trigger:** Arun opens AeroTwin's Historical Log view and selects the turbine blade component for the relevant simulation session.
Precondition: A CRITICAL alert was generated for Engine #4's turbine blade 3 days ago. The aircraft has been grounded for inspection. Arun needs to produce a root-cause analysis report.	**AeroTwin Features:** Historical log modal, sensor_readings database query, maintenance_log audit trail.

Main Flow:

1. Arun navigates to the Historical Log modal in AeroTwin.
2. He selects: Component = turbine_blade, Session = [date of incident], Time range = last 500 flight hours.
3. AeroTwin queries the sensor_readings and health_snapshots tables and returns the complete data history.
4. The trend graph shows a clear pattern: temperature and vibration were rising slightly above the degradation model's expected curve for the last 80 simulated flight hours before the anomaly.
5. The maintenance_log shows: an AMBER alert was generated 140 hours before the CRITICAL event, recommending inspection — which was not actioned in time in the simulation.
6. Arun can identify the exact flight hour at which the anomaly occurred, the sensor values at that moment, and the AeroTwin alert that was generated.
7. He exports the log data (CSV) for inclusion in the formal safety report.

Alternate Flow: If the historical data shows the AMBER alert was actioned (inspection completed), Arun can confirm the inspection was preventative and the CRITICAL event was a new, sudden fault rather than a missed gradual degradation.

Postcondition: A complete, time-stamped, tamper-evident record of the engine's degradation history is available for the safety audit — replacing the manual, incomplete paper log review process.

VALUE DELIVERED

Provides a complete, verifiable forensic record that supports both proactive safety management and post-incident investigation.

UC-05: Multi-Component Health Comparison for Maintenance Prioritisation

Actor: Line Maintenance Engineer (Rajesh Sharma)	**Trigger:** Rajesh has a limited 4-hour maintenance window available and needs to decide which component to prioritise.
Precondition: AeroTwin is monitoring three components simultaneously. At 350 simulated flight hours: turbine blade at 63% health (AMBER), bearing at 81% (GREEN), compressor at 54% health (AMBER).	**AeroTwin Features:** Multi-component health panel, comparative RUL display, fatigue acceleration indicator, maintenance recommendation engine.

Main Flow:

1. Rajesh opens the dashboard and views the multi-component health panel.
2. He sees all three components' health scores, fatigue trends, and predicted RULs side by side.
3. Turbine blade: RUL 280 hrs, rate of fatigue acceleration: moderate.
4. Bearing: RUL 480 hrs, rate of fatigue acceleration: slow.
5. Compressor: RUL 190 hrs, rate of fatigue acceleration: HIGH — the fastest degrading component.
6. The alert panel shows: 'Compressor — Priority inspection within 190 flight hours. Fatigue acceleration detected.'
7. AeroTwin recommends: 'If only one component can be inspected in the available window, prioritise the compressor based on lowest predicted RUL and highest fatigue acceleration rate.'
8. Rajesh uses the 4-hour window to inspect and service the compressor.

Alternate Flow: If Rajesh disagrees with the recommendation (e.g., operational context suggests the turbine blade is higher risk), he can override and log his reasoning in the maintenance_log.

Postcondition: Limited maintenance resources are allocated to the highest-risk component based on data-driven RUL comparison, rather than guesswork or fixed rotation schedules.

VALUE DELIVERED

Optimises limited maintenance resource allocation — a critical capability for airlines with tight turnaround schedules and limited engineering staff.

UC-06: Live Demonstration — Simulated Flight with Judge Interaction

Actor: Hackathon Judge / Evaluator	**Trigger:** A judge asks: 'Can you show me what happens when something goes wrong with the engine?'
Precondition: AeroTwin is running on a laptop in the demonstration area. The simulation is paused at baseline (all	**AeroTwin Features:** Full dashboard, anomaly injection, CRITICAL alert, historical log, live verbal Q&A guided by

components 100% healthy). AeroTwin's visible outputs.

Main Flow:

1. Demonstrator starts the simulation — sensor data begins streaming live on the dashboard.
2. Judge watches as the fatigue trend graphs build in real time over the next 60 seconds (representing ~30 simulated flight hours).
3. The bearing component transitions from GREEN to AMBER on the Three.js visualisation as its health drops to 72%.
4. An AMBER alert fires automatically: 'Bearing — Schedule inspection within 280 hours.'
5. Demonstrator clicks 'Inject Anomaly: Vibration Spike' in the control panel.
6. Within 2 seconds, the turbine blade mesh flashes RED, a CRITICAL alert appears: 'Anomalous vibration — Ground aircraft immediately.'
7. Judge asks: 'Where is this data coming from?' — Demonstrator explains the simulation engine and the NASA C-MAPSS calibration.
8. Judge asks: 'Can I see the history?' — Demonstrator opens the Historical Log modal showing the full degradation trajectory.
9. Judge asks: 'What would happen with a real engine?' — Demonstrator explains the IoT upgrade path (Arduino → Jetson Nano).

Alternate Flow: If a judge asks for a specific component to be stressed, the demonstrator can select which component's anomaly to inject from the control panel.

Postcondition: Judge has seen: real-time monitoring, automatic alerting, interactive anomaly simulation, historical audit trail, and a clear explanation of the hardware upgrade path — covering all key evaluation criteria.

VALUE DELIVERED

A compelling, interactive live demonstration that makes the system's capabilities immediately tangible to a non-technical or semi-technical judge.

AGILE USER STORIES

The following user stories are written in standard Agile format (As a [actor], I want [capability], So that [benefit]) and map directly to AeroTwin's implemented features. These stories define the acceptance criteria for the hackathon prototype:

ID	As a...	I want...	So that...	AeroTwin Feature
US-01	Line Maintenance Engineer	I want to see the real-time health score of each engine component on a single dashboard	I can immediately assess which components need attention without reading through raw	Health card + Three.js visualisation

			sensor logs.	
US-02	Fleet Operations Manager	I want the system to predict how many flight hours each component has remaining	I can schedule maintenance proactively during planned ground windows rather than reacting to failures.	Random Forest RUL prediction panel
US-03	Line Maintenance Engineer	I want to receive an automatic alert within seconds of an abnormal sensor reading	I can act on developing faults immediately rather than discovering them at the next scheduled check.	Z-score anomaly detector + Alert engine
US-04	Aviation Safety Officer	I want a complete, timestamped log of all sensor events and maintenance alerts	I have a verifiable audit trail for safety reviews and incident investigations.	SQLite maintenance_log + Historical Log modal
US-05	Line Maintenance Engineer	I want to compare the RUL of multiple components side by side	I can allocate limited maintenance resources to the highest-risk component first.	Multi-component RUL comparison panel
US-06	Fleet Operations Manager	I want to see a tiered severity system (GREEN / AMBER / RED / CRITICAL)	I can quickly prioritise my response without needing to interpret raw numbers.	Tiered alert severity system
US-07	Aviation Safety Officer	I want the system to flag anomalies that fall outside normal degradation patterns separately from gradual wear	I can distinguish between a predictable service requirement and an unexpected safety-critical event.	Z-score anomaly detector (separate from RUL model)
US-08	Line Maintenance Engineer	I want to view the historical sensor trend for any component over any past time window	I can understand how degradation built up over time and better plan future maintenance intervals.	Historical Log modal + /api/history endpoint